



מדעי המחשב

בחינת מתכונת – תשפ"א

משך הבחינה : 3 שעות ו-45 דק'.

מבנה השאלון ומפתח ההערכה: בשאלון זה שמונה שאלות המחולקות בין שלושה פרקים.

עליך להשיב על חמש שאלות סה"כ. משקלה של כל שאלה הוא 20 נק'.

פרק ראשון – בפרק זה שלוש שאלות.

פרק שני – בפרק זה שלוש שאלות.

פרק שלישי – בפרק זה שתי שאלות.

בשאלון הבגרות פרק זה מכיל שאלות בארבעה מסלולים שונים. בשאלון הנוכחי שתי שאלות במסלול "תכנות מונחה עצמים".

חומר עזר מותר בשימוש: כל חומר עזר חוץ ממחשב הניתן לתכנות.

נא לשמור על טוהר הבחינות.

שם: _____ כיתה: _____ מורה: _____

שלישי		שני			ראשון			פרק
8	7	6	5	4	3	2	1	שאלה
								בחרתי ב-

ההנחיות בשאלון זה מנוסחות בלשון זכר ומכוונות לנבחנות ולנבחנים כאחד.

בהצלחה!

פרק ראשון

שאלה 1 – 20 נקודות

כתוב את הפעולה *StrInArr* המקבלת שני פרמטרים: *str* – מחרוזת ו- *arr* מערך של תווים שונים זה מזה. הפעולה תחזיר אמת אם כל התווים של המחרוזת *str* מופיעים ברצף במערך *arr*. אחרת, הפעולה תחזיר שקר. למשל, עבור המחרוזת "and" ומערך התווים הבא הפעולה תחזיר אמת.

0	1	2	3	4	5	6
'x'	'o'	'a'	'n'	'd'	'z'	'\$'

שאלה 2 – 20 נקודות

להקת רוק מפורסמת יוצאת למסע הופעות בערי הבירה של מדינות אירופה. בכל עיר בירה תתקיימה שתי הופעות בדיוק. שתי ההופעות תערכנה באותו יום אך בשעות שונות. במהלך ההופעות, הצופים עומדים בשורות ממוספרות מול הבמה.

כרטיס הופעה (Ticket) מאופיין על ידי העיר בה מתקיימת ההופעה (*city*, מחרוזת), זמן ההתחלה של ההופעה (*time*), ממשי. למשל 22.5 משמעו שההופעה מתחילה בשעה 10 וחצי בערב), מחיר (*price*, ממשי), מספר השורה של הצופה (*row*, שלם גדול מ-0).

א. (2 נק') במחלקה **כרטיס הופעה** כתוב את כותרת המחלקה ותכונותיה. בנוסף, כתוב פעולה בונה מלאה המקבלת פרמטר עבור כל אחת מהתכונות ומעדכנת אותן.

ב. (5 נק') במחלקה **כרטיס הופעה** כתוב את הפעולה *Distance* המקבלת כפרמטר את *tic* - **כרטיס הופעה**. אם הכרטיס *tic* מתייחס לאותה ההופעה של הכרטיס הנוכחי (אותה עיר הופעה ואותו זמן התחלה), הפעולה תחזיר את המרחק בערך מוחלט בין השורה של הצופה שבכרטיס *tic* לבין השורה של הצופה שבכרטיס הנוכחי. אחרת, יש להחזיר -1.

ג. (13 נק') כתוב את הפעולה החיצונית *OtherTotalPrice* המקבלת כפרמטר את *arr* - מערך של **כרטיסי הופעה** וכפרמטר נוסף את *tic* - **כרטיס הופעה**. הפעולה תחזיר את סה"כ מחיר הכרטיסים ב-*arr* שאינם לאותה הופעה כמו הכרטיס *tic*. בפתרון לסעיף זה יש להשתמש בפעולה *Distance* שכתבת בסעיף ב'. הנח כי במחלקה **כרטיס הופעה** לכל תכונה מוגדרת פעולה מאחזרת (*Get*) וכי במערך *arr* אין הפניות ריקות (*null*).

שאלה 3 – 20 נקודות

באגסי-לנד היא מדינה של כפרי ארנבים כך שבכל **כפר ארנבים** מתגוררות 100 **משפחות ארנבים** לכל היותר.

משפחת ארנבים (RabbitFamily) מאופיינת על ידי שם המשפחה (*family*, מחרוזת) ומספר צאצאים (*offspring*, שלם). הנח כי במחלקה **משפחת ארנבים** מוגדרות פעולה בונה מלאה, פעולות מאחזרות (*Get*) לשתי התכונות ופעולה קובעת (*Set*) לתכונה *offspring*. בנוסף, מוגדרת הפעולה הבוליאנית *AtLeast8* המחזירה אמת אם במשפחה לפחות 8 צאצאים (ושקר אחרת).

כפר ארנבים (RabbitVillage) מאופיין על ידי שם הכפר (*name*, מחרוזת), מספר המשפחות המתגוררות בכפר (*count*, שלם בין 0 ל-100) ומשפחות הארנבים (*families*, מערך של משפחות ארנבים). הנח כי במחלקה **כפר ארנבים** מוגדרות פעולות מאחזרות (*Get*) לתכונות *name* ו-*count*. עוד הנח כי בעת יצירת **כפר ארנבים** הוא ריק ממשפחות.

א. (3 נק') במחלקה **כפר ארנבים** כתוב את הפעולה *AddFamily* המקבלת כפרמטר את *if* - **משפחת ארנבים**. הפעולה תוסיף את *if* למערך משפחות הארנבים אם יש מקום במערך. אחרת, תוצג הודעה מתאימה.

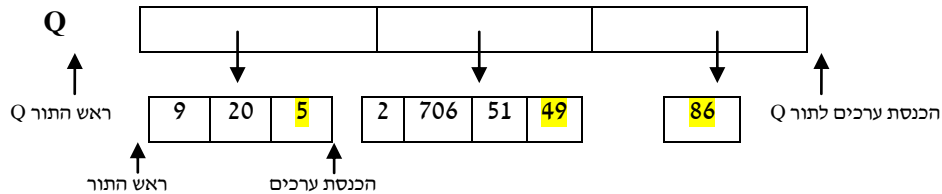
ב. (5 נק') במחלקה **כפר ארנבים** כתוב את הפעולה *BigF* שתחזיר לכמה מהמשפחות בכפר יש לפחות 8 צאצאים.

ג. (12 נק') כתוב את הפעולה החיצונית *PopulatedVillage* המקבלת כפרמטר את *vlgs* - מערך של **כפרי ארנבים**. הפעולה תחזיר את שמו של **כפר הארנבים** בו המספר הגבוה ביותר של משפחות בהן 8 צאצאים לפחות. הנח כי במערך *vlgs* אין הפניות ריקות (*null*) וכי בכל אחד מכפרי הארנבים מספר שונה של משפחות בהן 8 צאצאים לפחות.

פרק שני

שאלה 4 – 20 נקודות

- א. (17 נק') כתוב את הפעולה **LastVals** המקבלת כפרמטר את Q – תור בו כל איבר הוא מסוג תור של שלמים לא ריק. כלומר Q הוא **תור של תורים**. הפעולה תיצור ותחזיר שרשרת חוליות המכילה את הערך האחרון של כל אחד מהתורים של Q. בפתרון אין חובה לשמור על שלמות התורים.
- למשל**, עבור התור Q הבא, הפעולה תיצור ותחזיר את שרשרת החוליות הזו: 5->49->86->null



- ב. (3 נק') כתוב את סיבוכיות זמן הריצה של הפעולה **LastVals**. **הסבר קביעתך בפירוט.**

שאלה 5 – 20 נקודות

גולשים הרשומים באתר אינטרנט יכולים לנהל את שמות חיות המחמד שלהם. להלן ממשק שתי מחלקות הכולל פירוט תכונות ופעולות.

גולש רשום באתר - class Member	
private string userName	שם המשתמש של הגולש הרשום באתר
private Node<string> pets	רשימת שמות חיות המחמד של הגולש הרשום
public Member (string userName)	פעולה בונה – בעת יצירת גולש רשום אין לו חיות מחמד
public string GetUserName ()	פעולה מאחזרת לתכונה userName
public void AddPetName (string pn)	פעולה המוסיפה שם חיית מחמד לרשימה pets
public void Change (string pn, string pn2)	פירוט הפעולה בסעיף א' של השאלה

אתר האינטרנט - class Website	
private Node<Member> members	רשימת הגולשים הרשומים באתר
public Website()	פעולה בונה ריקה – בעת יצירת אתר הוא ריק מגולשים רשומים
public void AddMember (string un)	פעולה המוסיפה גולש רשום לרשימה members
public Member FindMember (string un)	אם ברשימה members יש גולש רשום ששמו un הוא יוחזר. אחרת יוחזר null
public void Swap (string A, string B, string p1, string p2)	פירוט הפעולה בסעיף ב' של השאלה

- א. (14 נק') במחלקה **Member** כתוב את הפעולה **Change** המקבלת כפרמטר את pn – מחרוזת המייצגת שם של חיית מחמד, המופיע פעם אחת בדיוק ברשימה pets ופרמטר נוסף pn2 – מחרוזת המייצגת שם של חיית מחמד שאינו מופיע ברשימה pets. הפעולה תמצא ברשימה pets את החוליה שערכה הוא pn. אם ערך הפרמטר pn2 הוא null, החוליה שערכה הוא pn תוסר מן הרשימה. אחרת ערך החוליה יעודכן להיות pn2.

- ב. (6 נק') במחלקה **Website** כתוב את הפעולה **Swap** המקבלת כפרמטרים את A ו-B – מחרוזות שונות זו מזו המייצגות שמות משתמש של שני גולשים הרשומים באתר, הפרמטר p1 – מחרוזת המייצגת שם הקיים ברשימת שמות חיות המחמד של הגולש הרשום A, והפרמטר p2 – מחרוזת המייצגת שם הקיים ברשימת שמות חיות המחמד של הגולש הרשום B.

ברשימת שמות חיות המחמד של הגולש הרשום A הפעולה תחליף את p1 ב-p2. באופן דומה ברשימת שמות חיות המחמד של הגולש הרשום B הפעולה תחליף את p2 ב-p1. **בפתרון עליך להשתמש בפעולות המתאימות מבין הפעולות המפורטות בטבלאות שלעיל. הנח כי הן מומשו עבורך.**

- ג. (2 נק' בונוס) כתוב את סיבוכיות זמן הריצה של הפעולה **Swap**. נמק קביעתך תוך ניתוח סיבוכיות זמן הריצה של כלל הפעולות המעורבות בפתרון.

נתונה פעולה רקורסיבית **What** המקבלת כפרמטרים שני מספרים שלמים גדולים מ-0:

```
public static int What(int x, int y)
{
    if(y == 0)
        return 0;

    if(y % 2 == 0)
        return What(x + x, y / 2);

    return What(x + x, y / 2) + x;
}
```

א. (4 נק') רשום מעקב רקורסיבי מלא אחר הזימון הבא:

int r1 = **What** (3, 5);

ב. (1 נק') כתוב מהו ערכו של המשתנה r2 לאחר הזימון הבא (אין צורך לרשום מעקב).

int r2 = **What** (3, 4);

ג. (1 נק') כתוב את טענת היציאה של הפעולה **What** - כלומר, הסבר מה הפעולה מבצעת.

נתונה פעולה רקורסיבית **Why** המקבלת כפרמטרים את a - מערך שלמים גדולים מ-0 ואת n - שלם לא שלילי וקטן מאורך המערך a:

```
public static int Why(int[] a, int n)
{
    int p = a.Length - 2;

    if (n == p)
        return What(a[p], a[p + 1]);

    int med = Why(a, n + 1);

    int res = What(a[n], a[n + 1]);

    return Math.Max(med, res);
}
```

נתון המערך arr הבא:

0	1	2	3
7	2	8	11

ד. (5 נק') רשום מעקב רקורסיבי מלא אחר הזימון הבא:

int r3 = **Why**(arr, 0);

שים לב: במעקב בסעיף זה אין צורך לעקוב אחר הפעולה **What** אלא לציין את הערך המוחזר ממנה בלבד.

ה. (1 נק') כתוב את טענת היציאה של הפעולה **Why** - כלומר, הסבר מה הפעולה מבצעת.

ו. (8 נק') **סעיף זה אינו קשור אל הסעיפים הקודמים של השאלה.**

כתוב פעולה רקורסיבית **UpPairs** המקבלת כפרמטר את arr - מערך של שלמים גדולים מ-0.

הפעולה תציג את כל זוגות המספרים הסמוכים במערך עבורם מתקיים כי המספר הראשון בזוג קטן מהמספר שלאחריו.

למשל, עבור המערך הבא הפעולה תציג את הזוג (2, 8) ואת הזוג (8, 11).

0	1	2	3	4
7	2	8	11	9

פרק שלישי

שאלה 7 – 20 נקודות

חברה מוכרת מכוניות ואופנועים. להלן ממשק המחלקות **מכונית (Car)** ו**אופנוע (Bike)** הכולל פירוט תכונות ופעולות.

class Car - מכונית		
private string model	דגם המכונית	תכונות
private double bPrice	מחיר בסיסי של מכונית מדגם זה	
private bool electric	מנוע חשמלי (אמת) או רגיל (שקר)	
private bool automatic	תיבת הילוכים אוטומטית (אמת) או ידנית (שקר)	
public Car (string m, double bP, bool e, bool a)	פעולה בונה המאתחלת את כל התכונות	פעולות
public double FinalPrice ()	מחשבת ומחזירה את המחיר הסופי של המכונית המורכב ממחיר הבסיס + תוספת של 20% ממחיר הבסיס לתיבה אוטומטית + תוספת של 10% ממחיר הבסיס למנוע חשמלי	
public string GetModel()	פעולות קובעות ומאחרות לתכונות	
public void SetModel (string model)		
public double GetBPrice ()		
public void SetBPrice (double bPrice)		
public bool GetElectric ()		
public void SetElectric (bool electric)		
public bool GetAutomatic ()		
public void SetAutomatic (bool automatic)		

class Bike - אופנוע		
private string model	דגם האופנוע	תכונות
private double bPrice	מחיר בסיסי של אופנוע מדגם זה	
private bool electric	מנוע חשמלי (אמת) או רגיל (שקר)	
private bool street	אופנוע כביש (אמת) או שטח (שקר)	
public Bike (string m, double bP, bool e, bool s)	פעולה בונה המאתחלת את כל התכונות	פעולות
public double FinalPrice ()	מחשבת ומחזירה את המחיר הסופי של האופנוע המורכב ממחיר הבסיס + תוספת של 15% ממחיר הבסיס לאופנוע שטח + תוספת של 10% ממחיר הבסיס למנוע חשמלי	
public string GetModel ()	פעולות קובעות ומאחרות לתכונות	
public void SetModel (string model)		
public double GetBPrice ()		
public void SetBPrice (double bPrice)		
public bool GetElectric ()		
public void SetElectric (bool electric)		
public bool GetStreet ()		
public void SetStreet (bool street)		

- א. (7 נק') נרצה להגדיר את המחלקה **כלי רכב (Vehicle)** ממנה יורשות המחלקות **מכונית ואופנוע**. כתוב תרשים UML עבור המחלקות **Vehicle, Car ו-Bike**. לכל מחלקה בתרשים ציין את התכונות וכותרות הפעולות באופן המתאים ביותר לעקרונות ההורשה והפולימורפיזם.
- ב. (5 נק') ממש את הפעולה **FinalPrice** בכל המחלקות בהן היא מופיעה.
- ג. (8 נק') כתוב פעולה חיצונית בשם **OffroadAndElectric** המקבלת כפרמטר את **vs** - מערך של **כלי רכב**. הפעולה תחזיר את סכום המחיר הסופי של כל אופנועי השטח החשמליים במערך **vs**. הנח כי במערך **vs** אין הפניות ריקות (**null**).

לפניך המחלקות A ו-B.

```

public class A
{
    private int x;
    public A() { this.x = 5; }
    public A(int x) { this.x = x; }
    public virtual int GetX() { return x; }
    public void DoSomething() { this.x = 1 + this.GetX(); }
    public virtual void Add() { this.x = this.x + 1; }
    public override string ToString(){ return "x in A: " + this.x; }
}
public class B : A
{
    private int x;
    public B() : base() { this.x = 2; }
    public B(int x) : base(x) { this.x = x - 2; }
    public B(int x1, int x2) : base(x1) { this.x = x2; }
    public override int GetX(){ return x; }
    public override void Add() { this.x = this.x + 5; }
    public int FlowerOfBase() { return base.GetX(); }
    public override string ToString() { return base.ToString() + " x in B: " + this.x; }
}

```

א. 9 נק' שרטט את העצמים הנוצרים בקטע התכנית הבא. לכל עצם ציין את סוגו וערכי תכונותיו.

ב. 2 נק' כתוב את הפלט של קטע התכנית.

```

A a1 = new A(2);
Object[] objs = new Object[6];
objs[0] = new A();
objs[1] = new A(7);
objs[2] = new B();
objs[3] = new B(7);
objs[4] = new B(0, 2);
objs[5] = a1;

for (int i = 0; i < 6; i++)
    Console.WriteLine(objs[i]);

```

ג. 9 נק' נתון קטע תכנית נפרד.

הצב כל אחד מזוגות ההוראות המסומנים ב-1 עד 3 במקום הסימון הבא **////****

במידה וקטע התכנית ירוץ באופן תקין כתוב מהו הפלט שיוצג.

אחרת, ציין את סוג השגיאה/ות (הידור/קומפילציה, ריצה) והסבר מדוע ההוראה/ות שגויה/ות.

```

A a2 = new A(2);
A a3 = new B(3, 5);
////**

```

a3.DoSomething();	}	זוג 1
Console.WriteLine(a3);		
((B)a2).Add();	}	זוג 2
Console.WriteLine(a3.FlowerOfBase());		
B b = (B)a3;	}	זוג 3
Console.WriteLine(b.FlowerOfBase());		